



Caruca v2

LLM-based specification mining for opaque shell commands

Caruca v2: Project Status Report

Interim findings for Prof. Eiers and Prof. Greenberg,
22 September 2026

SOURCE `ai_docs/analysis/status_report_2026-09-22.md`

GENERATED September 22, 2026 at 12:55 PM

PAGES 14

CARUCA V2 — RESEARCH PROJECT

Table of contents

Caruca v2: Project Status Report

1. In one paragraph
 2. Terms used in this report
 3. Where the project stands
 4. Interim findings
 5. Problems, and what we did about them
 6. What this adds to the paper
 7. Proposed changes to the case study, and what waits for a follow-up
 8. Questions for this meeting
 9. What is left to do
 10. Further reading
-

Caruca v2: Project Status Report

To: Prof. William Eiers, Prof. Michael Greenberg

From: Tiran Dagan

Date: 22 September 2026

Status: interim. Every number below is reproducible from committed run records at Caruca commit `d8032407` , model `gpt-4o` , temperature 0, three samples per cell.

1. In one paragraph

The question is whether a language model, given the same inputs Caruca uses, can stand in for each of Caruca's four hand-written stages, and what that costs. We built a model-driven copy of the full pipeline (v2), a measurement harness that scores the original (v1) and v2 with the same instrument, and ran a head-to-head on nine commands. The answer so far is a distinction, not a score: **the model reproduces what Caruca writes far better than what Caruca means**. It recovers every flag and almost every invocation string, then asks for the wrong test environment, fails to report what it observed during execution, and cannot read the largest traces at all. Total model spend for the study was \$3.50. Caruca's side of the same work took 17 seconds and no API calls.

2. Terms used in this report

TERM	PLAIN MEANING
Specification	A structured description of a command: its flags, the type of value each takes, and what the command does to files. PaSh, POSH and ShellCheck consume these.
The four stages	1: read the manual page and write the syntax specification. 2: turn the specification into concrete command invocations, each with the files it needs. 3: run each invocation in a sandbox and record what it touched. 4: turn those records into the specification a consumer like PaSh reads. In Caruca only stage 1 uses a model; v2 uses a model for all four.
Flag recall	Of the flags in the reference specification, the share the other side also found. 1.0 means nothing was missed.
Invocation recall	Of the distinct command lines Caruca generates, the share v2 also generated.
Environment agreement	Of the test environments v2 asked for, the share Caruca would also have built for that command line. A correctness rate, not a reach figure.
Recovery	Of the file interactions Caruca's tracer recorded, the share v2 also reported.
Coverage	Reserved for the paper's meaning only: how much of real-world usage a specification reaches. Not yet measured for v2.
Bound	A cap on how many optional elements one invocation may carry. The study used a bound of one on both sides to keep costs low. The paper's default is four.
Temperature 0	The model setting intended to make it give the same answer to the same question every time. It does not fully do so.
Context window	The maximum text a model can read in one request. For gpt-4o, about 128,000 tokens.

3. Where the project stands

BUILT AND WORKING	STATE
v2 pipeline, all four stages, as a command-line tool	done, 163 recorded runs
Naive single-prompt control (Prof. Eiers' baseline)	done; same code path as stage 1
Measurement harness: one scorer per stage, campaign runner, cost and token telemetry, repeat-sampling	done, 261 tests; every scorer proves itself by scoring Caruca against itself and requiring a perfect result
Linux virtual machine on the Mac for Caruca's tracing and annotation	done, see §5
Artifact provenance check on the paper's grep, ps and cp claims	done
Nine-command parity study, all four stages	done, written up
Draft addendum to the paper, organized by the paper's own sections	drafted for discussion
Execution console (a local web page to run and inspect both systems)	specified only, not built
Improvement program (task 011): every proposed change to v2 sorted into one of five kinds before it is made	diagnosis done, no code changed

4. Interim findings

Nine commands (`cat` , `pwd` , `rm` , `sha256sum` , `tac` , `tail` , `tee` , `uniq` , `wc`), three runs each, both systems scored by the same instrument against the same reference.

STAGE	VERDICT	THE NUMBER	ONE REAL EXAMPLE
1. Syntax specification	Replicates	Flag recall 1.0 on both sides. Arguments typed exactly right: Caruca 98.3%, v2 96.7%.	<code>tail</code> : v2 lists <code>--follow</code> but drops its short spelling <code>-f</code> , the form most people type. Caruca keeps both.
2. Configuration generation	Diverges	Invocation recall 87.8%. Environment agreement 20.8%. 146 of 285 invocations exceed the bound.	<code>grep a relpath_1</code> : v2 types the pattern <code>a</code> as a file that must already exist, so the sandbox is built with a file named <code>a</code> in it. Caruca types it as needing nothing. Same command line, valid configuration, wrong world.
3. Execution and tracing	Does not replicate	105 of 135 sessions (78%) never reported. Where reported, recovery 60.6%. 12 of 27 cells usable.	For <code>wc</code> , in all 15 sessions the model ran the command once, then described the result in prose instead of calling the reporting tool it was given, and stopped with 13 of 15 turns unused.
4. Annotation	Diverges, with a hard ceiling	Class agreement with ground truth: v2 25 of 77 (32%), Caruca 20 of 83 (24%). 6 of 27 cells impossible.	<code>rm</code> : its trace is about 211,000 tokens, larger than the 128,000 the model can read, so v2 produces nothing. Caruca's annotator processes the same file in under a second.

What the pattern means. v2 reproduces what Caruca writes (flags, command lines) far better than what Caruca means (which files must exist, what actually happened when the command ran). The stage-2 example is the important one: nothing downstream objects to the wrong environment, so the error would propagate silently into every later stage. Stage 3 is a behaviour finding, not an accuracy finding: the model does not follow a reporting protocol it was given, and it fails on some commands every time and on others almost never. Stage 4 is the first limit that belongs to the approach rather than to our setup, and it scales the wrong way: the more thoroughly a command is traced, the less of the result a model can read.

Remediation ideas, per stage. Each is labelled with the kind of change it is under the task 011 rule (section 5): a *measurement fix* changes only the scorer, a *request repair* gives the model a rule it was never told, an *alignment* gives v2 exactly what Caruca had, and a *scope change* is new behaviour that Tiran decides and that runs as a separate measured arm beside the unchanged one.

STAGE	IDEA	KIND
1	Send Caruca's exact prompt, captured from its code, instead of a paraphrase (task 009).	alignment
1	Count missing and extra spellings inside a matched flag, so a dropped <code>-f</code> shows up as missing rather than hiding inside a matched <code>--follow</code> .	measurement fix
2	State the bound in the prompt as Caruca's code enforces it (an optional file operand counts as one of the optional elements), not as its help text describes it. This alone accounts for 403 of the 414 extra command lines.	request repair
2	Explain every path type in the request, including the "already exists" type the schema never describes, with one example each.	request repair
2	Compare environments by which files would exist, not by how the setup is written down. Removes about half of the mismatches.	measurement fix
2	Ask once for every environment variant Caruca would enumerate for an invocation.	scope change

STAGE	IDEA	KIND
3	Tighten the instructions: “Do not explain or discuss the results. Do not answer in prose. Your only output is a call to the reporting tool, with the observations in the JSON shape given.”	scope change
3	Require the tool call at the API level, so a prose answer is not an accepted way to end a turn.	scope change
3	If a session ends without a report, ask once: “You have not reported your observations. Call the reporting tool now.” This is the stage-2 nudge applied to stage 3.	scope change
3	Record per command which of the above closes the gap, since failure was command-dependent (<code>pwd</code> 13 of 15 reported, <code>wc</code> 0 of 15).	measurement
4	Give the model the same relevance-filtered trace Caruca’s own annotator reads, not the raw one (<code>ls</code> : 17,512 raw records reduce to 20). Whether Caruca’s annotator reads the filtered form is an open check.	alignment, pending check
4	Annotate in pieces: one call per configuration or per flag group, then merge, so no single request exceeds the window.	scope change
4	Run a model with a larger window (Claude’s 200,000 tokens or more) as a measured arm, to separate “the approach cannot” from “this model cannot”.	scope change

The stage-3 prompt tightening is the cheapest test and the one most likely to move the headline. It is still a scope change: the naive control was deliberately not told how to behave beyond naming the tool, and every instruction added runs in the direction of flattery. So it runs as an arm, and both numbers are reported.

Two smaller findings. Temperature 0 did not give repeatable answers: `tac` varied across runs at stage 1, and `pwd` at stage 3, so every model step must be run more than once. And Caruca’s cautious direction is consistent: of its 63 class disagreements with the human annotators, 58 are it being more conservative, which is the right way to be wrong when evidence is thin. Two stage-2 causes are now diagnosed: 403 of v2’s 414 extra command lines come from the prompt stating the bound as Caruca’s documentation describes it rather than as its code enforces it, and about half the environment mismatches are notation rather than substance, leaving one real residual, a path type the schema never explains.

A reproducibility finding about the artifacts. Caruca's committed model output, scored against its committed ground truth with Caruca's own comparison script, gives 78 of 116 exact, against the paper's 116 of 120. An independent scorer gives 83 of 116, and the ground truth scored against itself is perfect, so the instrument is sound. The likely explanation is that the committed output is a different run from the one the paper reported. A question for the authors, not a claim that the paper is wrong.

Correction note: the parity study document quotes v2's stage-4 agreement from its first run only (10 of 33) and Caruca's disagreement count as 62. Both were re-checked against the stored results today; the figures above are the corrected ones and the documents will be updated.

5. Problems, and what we did about them

- **Caruca's tracing and annotation cannot run on macOS.** Tracing needs Linux kernel features, and annotation fails on the Mac because `/tmp` resolves to a different path than the code expects. **Solution:** a dedicated Linux virtual machine on the Mac (Lima, Ubuntu 24.04) with a kernel setting fix that makes unprivileged sandboxes work. Both systems now trace in the same VM on the same binaries, and v2 can hand any comparison to Caruca inside it. Destructive commands such as `rm` run there for v2 too.
- **Caruca's data schema understates what Caruca requires.** Three fields validate any value while the code accepts only a few. The model was therefore asked questions it could not answer correctly, and validation gave false assurance. **Solution:** complete the request with the real constraints before running. We treat this as repairing the instrument, not tuning the model, and we record the line between the two.
- **Naive comparisons reported Caruca disagreeing with itself.** Its outputs contain random identifiers and unordered sets. **Solution:** every scorer runs Caruca twice and must find perfect agreement before it is trusted on v2.
- **Our own numbers needed correcting three times** (an environment figure that counted undefined cells as zero, a stage-3 figure computed by hand over half the data, and a claimed Caruca enumeration defect that turned out to be a v2 error). **Solution:** every correction stays visible in the document with the old value, and the console specification adds an automatic re-check of every published number against its evidence.
- **v2's stage-1 prompt was a paraphrase of Caruca's.** That was the largest uncontrolled difference at stage 1. **Solution:** the licensing question is now settled, so the next campaign sends Caruca's exact prompt, captured from Caruca's own code, and reports whether the small typing gap closes. Two facts surfaced on the way: Caruca's prompt was edited twice after its committed specifications were generated, and v2 drops one spelling of `tail`'s follow flag, which the scorer had not counted.
- **The risk of improving v2 by teaching to the test.** Once a gap is measured, every fix is a temptation to tune against the nine commands. **Solution:** a written rule that sorts every proposed change into one of five kinds before it is made: a scorer fix, a repair to a request the model could not have answered, an alignment to what Caruca actually had, a scope change that only Tiran decides and that runs as a separate measured arm, or tuning, which is excluded. This is the discipline that keeps the naive control honest.

6. What this adds to the paper

These are additions the original evaluation could not make because the measurements did not exist. Each is framed as strengthening the paper's case.

1. **The model step has never been costed.** The paper reports hours per command for tracing but no tokens, dollars or seconds for syntax inference. We now have those numbers, and they are small. The honest framing is not “model versus free” but “model versus seconds”.
2. **Accuracy as a distribution, not a single run.** Temperature 0 gave different answers on one command in nine. Reporting a median and range costs cents and converts an unqualified number into a measured one.
3. **A specification can be valid and still mean the wrong thing.** Stage 2 shows a configuration can pass every check and describe the wrong world. The paper's evaluation checks syntax against ground truth and derived specifications against consumers, but nothing checks the environments in between. A cheap environment-level check would close that gap.
4. **An empirical answer to “why not just use a model?”** Reviewers will ask. We can now say which stages a model can stand in for (stage 1), partly (stage 2), and cannot (stages 3 and 4), with the context-window ceiling as a concrete limit that scales the wrong way: the more thoroughly a command is traced, the less a model can read the result. This is the strongest argument in the architecture's favour that the work has produced.
5. **A bound-semantics note.** The optional-argument cap counts file operands as well as flags, so at any setting a command with an optional file argument gets one fewer flag than the documentation implies. Whether that is intended is a question for the authors; either answer is a one-sentence clarification.

7. Proposed changes to the case study, and what waits for a follow-up

Within the original goals (compare Caruca to a model-based pipeline on correctness, cost, coverage, consistency, and hand-written code replaced):

- Run the same nine commands at the paper's two-flag bound, which the paper's own timings suggest is affordable, so the comparison sits on the paper's ground rather than a cost-driven one.
- Re-run stage 1 with Caruca's exact prompt, and with the dated model version Caruca used if it is still served, to separate “same prompt” from “same model”.

- Add a second model as a measured arm. A pilot showed Claude Haiku 4.5 costing 10 to 50% less per cell with equal stage-1 results.
- Add the real-world coverage measurement for v2. Caruca's oracle only reads its own specifications, so a small harness change is needed before v2 can be scored on the 665,628-line invocation corpus.
- Add Caruca-side telemetry (tokens and per-step timing) so cost is measured on both sides rather than one.

For a follow-up study, so the current one stays a clean control:

- An agentic arm: the same stages driven by a tool-using agent rather than one prompt, with the same information budget, to measure what the agent harness buys. Already specified, not run.
- Whether a single “did you report everything?” prompt fixes the stage-3 reporting failure, and what that does to every other number. Deliberately not done here because each nudge runs in the direction of flattery.
- Running the actual consumers (PaSh, POSH, ShellCheck) on v2's annotations, which is the paper's first evaluation question and a heavier task.
- Storing the real captured command output next to the model's retyped copy in stage 3, to measure copying fidelity. Checked on 19 cases so far with no errors, but the sample is small.

8. Questions for this meeting

1. Which run produced the paper's 116 of 120 figure, and can that artifact be committed? (Michael)
2. Is a file operand meant to count against the optional-element cap? (Michael)
3. Where did the paper's `ps` run come from, and which revision produced the `cp` result? The current annotation set contradicts both. (Michael)
4. Should the stage-3 reporting failure be left as the result, or is forcing the reporting tool call within scope, as a separate measured arm? The same question applies to asking stage 2 for every environment variant. (both)
5. Is the level of intervention on the naive control still what you intended? Everything changed so far is a repair to the request, never to the answer. (William)

9. What is left to do

ITEM	EFFORT	BLOCKED ON
Stage 1 rerun with Caruca's verbatim prompt (task 009)	small, about \$2	cost approval
Improvement program, stages 1 to 4 (task 011)	moderate, about \$10	cost approval and three scope decisions: force the reporting tool call in stage 3, ask for every environment variant in stage 2, switch stage 1's default prompt
Apply today's figure corrections to the parity study and addendum	trivial	none
Two-flag bound rerun of the parity study	moderate	decision
Execution console, replay and inspection first (task 010)	large, no spend	four design decisions
Caruca-side telemetry (task 005)	small	none
Agentic arm (task 007)	moderate	its own cost line
Coverage instrument for v2	small	none
Move every finding into one file per finding, with evidence and paper section	moderate	console phase 3
Correct three documents that still name the wrong invocation corpus file	trivial	none

10. Further reading

Supporting documents, by name: `v1_v2_parity_study` (the full comparative report with every confound), `white_paper_addendum` (eight proposals in the paper's section order), `e0_artifact_pinning` (provenance of the grep, ps and cp claims), `v2_fidelity_to_v1` (mechanism-by-mechanism account of what v2 reproduces and where it deliberately differs), and `v2_chain_vs_v1` (one command followed through both pipelines).